

資料探勘 Homework 3 完整報告

摘要

本報告記錄了資料探勘課程第三次作業的完整流程，從資料前處理到模型提交的全過程。更新後所有 submission CSV 欄位皆符合 sample submission 的 **Id**, **Action** 格式；最終在 Kaggle 平台上以 **v2_balanced** 取得最佳成績：**Private Score: 0.67450, Public Score: 0.67875**。

(a) 資料前處理

1. 資料集概述

項目	數值
訓練集大小	32,769 筆
測試集大小	58,921 筆
特徵數量	8 個
目標變數	ACTION (0 或 1)

2. 特徵欄位分析

欄位名稱	說明	唯一值數量	類型
RESOURCE	資源 ID	7,518	ID
MGR_ID	管理者 ID	4,243	ID
ROLE_ROLLUP_1	角色階層 1	128	ID
ROLE_ROLLUP_2	角色階層 2	177	ID
ROLE_DEPTNAME	部門名稱	449	ID
ROLE_TITLE	職稱	343	ID
ROLE_FAMILY_DESC	角色家族描述	2,358	ID
ROLE_FAMILY	角色家族	67	ID
ROLE_CODE	角色代碼	343	ID

觀察: 所有特徵皆為 ID 類型（類別變數的編碼），屬於高基數類別特徵。

3. 資料品質檢查

3.1 缺失值檢查

- 訓練集缺失值: 0
- 測試集缺失值: 0

3.2 資料類型

- 所有特徵皆為數值型 (int64)，無需編碼轉換

4. 目標變數分佈 (類別不平衡問題)

ACTION 值	數量	比例
1	30,872	94.21%
0	1,897	5.79%

問題診斷: 資料存在嚴重的類別不平衡問題，比例約為 **16:1**

5. 類別不平衡處理實驗

我們測試了多種資料不平衡處理方法：

方法	準確率	F1-Score	AUC-ROC	少數類別召回率
基準模型 (無處理)	94.87%	0.9733	0.8610	26.65%
SMOTE 過採樣	94.45%	0.9708	0.8478	35.88%
Random Undersampling	76.98%	0.8635	0.8247	72.30%
SMOTETomek	94.43%	0.9707	0.8472	38.26%
class_weight='balanced'	94.72%	0.9722	0.8628	39.84%
SMOTE + class_weight	94.45%	0.9708	0.8478	35.88%

結論: `class_weight='balanced'` 在保持高準確率與 F1-Score 的同時，顯著提升少數類別召回率 (26.65% → 39.84%)，因此選擇此方法作為最終方案。

(b) 模型訓練流程

1. 模型選用

我們測試了多種模型：

模型	驗證集準確率
Random Forest	94.72%
Gradient Boosting	94.19%
Logistic Regression	94.22%
Decision Tree	92.74%

最終選擇: Random Forest - 在準確率和訓練效率之間取得最佳平衡

2. 參數設定

2.1 預設參數 (v1)

```
RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
```

2.2 調優後參數 (v2, GridSearchCV scoring='f1')

```
RandomForestClassifier(  
    n_estimators=200,  
    max_depth=25,  
    min_samples_split=5,  
    min_samples_leaf=1,  
    random_state=42,  
    n_jobs=-1  
)
```

2.3 Threshold tuning 參數 (v3)

```
RandomForestClassifier(  
    n_estimators=200,  
    max_depth=25,  
    min_samples_split=5,  
    min_samples_leaf=1,  
    random_state=42,  
    n_jobs=-1  
)  
  
# 使用驗證集搜尋最佳 threshold  
best_threshold = 0.41
```

2.4 集成模型參數 (v4)

```
# Random Forest  
RandomForestClassifier(n_estimators=200, max_depth=20, min_samples_split=5,  
    min_samples_leaf=1, random_state=42, n_jobs=-1)  
  
# Gradient Boosting  
GradientBoostingClassifier(n_estimators=150, max_depth=5, learning_rate=0.1,  
    random_state=42)  
  
# Logistic Regression  
LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced')
```

3. 網格搜尋參數空間 (GridSearchCV)

```
param_grid = {  
    'n_estimators': [150, 200],  
    'max_depth': [15, 20, 25],  
    'min_samples_split': [3, 5],  
    'min_samples_leaf': [1, 2]  
}
```

最佳參數: {'n_estimators': 200, 'max_depth': 25, 'min_samples_split': 5, 'min_samples_leaf': 1}

(c) 模型評估

1. 內部驗證集評估

Submission	驗證集準確率	F1-Score	AUC-ROC
v1	94.72%	0.9723	0.8390
v2	94.87%	0.9733	0.8610
v3	94.95%	0.9738	0.8610
v4	94.19%	0.9700	0.7580

2. Cross-Validation 結果

- **v2 (GridSearchCV, scoring='f1')**: 交叉驗證 F1-Score = **0.9741**
- **v2_balanced (GridSearchCV, scoring='f1')**: 交叉驗證 F1-Score = **0.9735**

3. Kaggle 評分結果

3.1 原始版本

Submission	修改內容	Private Score	Public Score
第一次 (v1)	基礎 Random Forest (n_estimators=100)	0.67073	0.67498
第二次 (v2)	GridSearchCV 調參, scoring='f1'	0.64071	0.65635
第三次 (v3)	Random Forest + threshold tuning	0.60626	0.61920
第四次 (v4)	特徵工程 + 集成模型	0.58913	0.58772

3.2 平衡版本 (class_weight='balanced')

Submission	修改內容	Private Score	Public Score
第一次 (v1_balanced)	基礎 RF + class_weight	0.65628	0.66703
第二次 (v2_balanced)	GridSearchCV + class_weight, scoring='f1'	0.67450	0.67875

Submission	修改內容	Private Score	Public Score
第三次 (v3_balanced)	RF + class_weight + threshold tuning	0.65289	0.65785
第四次 (v4_balanced)	集成模型 + class_weight	0.62636	0.62442

(d) 模型改進流程

1. 每次 Submission 的修改內容

Submission 1 (v1)

- **修改內容:** 使用預設參數的 Random Forest 作為 baseline
- **模型參數:** `n_estimators=100`, `random_state=42`
- **Kaggle Score:** Private 0.67073 / Public 0.67498

Submission 2 (v2)

- **修改內容:** 使用 GridSearchCV 進行參數調優, scoring 改為 F1-Score
- **模型參數:** `n_estimators=200`, `max_depth=25`, `min_samples_split=5`, `min_samples_leaf=1`
- **驗證結果:** Accuracy 0.9487 / F1 0.9733 / AUC 0.8610
- **Kaggle Score:** Private 0.64071 / Public 0.65635

Submission 3 (v3)

- **修改內容:** 使用 Random Forest 預測機率, 並在驗證集上進行 threshold tuning
- **模型參數:** `n_estimators=200`, `max_depth=25`, `min_samples_split=5`, `min_samples_leaf=1`
- **最佳 threshold:** 0.41
- **驗證結果:** Accuracy 0.9495 / F1 0.9738 / AUC 0.8610
- **Kaggle Score:** Private 0.60626 / Public 0.61920

Submission 4 (v4)

- **修改內容:** 加入特徵工程 (ID出現次數) + 集成 Random Forest、Gradient Boosting、Logistic Regression
- **Kaggle Score:** Private 0.58913 / Public 0.58772
- **變化:** 分數大幅下降 (-0.11655 / -0.11676)

2. 修改原因分析

Submission	修改原因	結果
v1	建立 baseline	基準分數 0.67
v2	以 F1-Score 進行 RF 參數調優	Private 0.64071 / Public 0.65635
v3	調整預測 threshold	驗證 F1 提升, 但 Kaggle 降至 Private 0.60626 / Public 0.61920
v4	嘗試特徵工程與集成學習	分數大幅下降, 方法不適合

3. Kaggle Score 變化分析

3.1 v2 改用 F1-Score 的原因

- **原因 1:** 資料不平衡下 accuracy 容易偏向多數類別, F1-Score 更能反映分類品質
- **原因 2:** 最佳參數與原本一致, 但交叉驗證指標改為 $F1=0.9741$, 評估目標更符合不平衡資料
- **Kaggle 結果:** 原始 v2 分數為 Private 0.64071 / Public 0.65635; 加入 `class_weight='balanced'` 的 v2_balanced 提升至 Private 0.67450 / Public 0.67875, 顯示類別權重比單純調整 scoring 更有效

3.2 v3 threshold tuning 的效果

- **關鍵因素:** 將預設 threshold=0.5 調整為 0.41
- **F1-Score 提升:** v2 的 0.9733 → v3 的 0.9738
- **Kaggle 結果:** 原始 v3 下降至 Private 0.60626 / Public 0.61920; v3_balanced 也低於 v2_balanced, 為 Private 0.65289 / Public 0.65785
- **結論:** threshold tuning 雖可提升驗證集 F1-Score, 但未能改善 Kaggle 分數, 推測 threshold 對驗證集過度調整, 泛化效果不佳

3.3 為什麼 v4 分數大幅下降?

- **可能原因 1:** 特徵工程 (ID出現次數) 可能引入雜訊
- **可能原因 2:** 集成模型中各模型權重未優化
- **可能原因 3:** 過擬合於驗證集, 導致泛化能力下降
- **教訓:** 複雜模型不一定更好, 簡單模型 + 正確的資料處理更有效

4. 最佳模型總結

項目	數值
最佳 Kaggle Submission	v2_balanced
模型	Random Forest
關鍵方法	<code>class_weight='balanced'</code> + GridSearchCV (<code>scoring='f1'</code>)
最佳參數	<code>n_estimators=200, max_depth=25, min_samples_split=3, min_samples_leaf=1</code>
Private Score	0.67450
Public Score	0.67875

結論與建議

關鍵發現

1. **類別不平衡是關鍵問題:** 本資料集存在 16:1 的類別不平衡, 直接影響模型在測試集上的表現
2. **class_weight 比 threshold tuning 更有效:** v2_balanced 是最佳 Kaggle 結果, 而 threshold tuning 雖提升驗證 F1, Kaggle 分數反而下降
3. **驗證集分數不可靠:** 驗證集上的高分不代表測試集上也會高分, 需要搭配 Kaggle 分數與交叉驗證綜合判斷

未來改進方向

1. **更細緻的類別權重調整**: 可以嘗試自定義 `class_weight` 比例
 2. **特徵工程優化**: 考慮使用目標編碼 (Target Encoding) 而非計數特徵
 3. **模型融合策略**: 學習更複雜的集成方法, 如 Stacking
 4. **更多交叉驗證**: 使用更多 folds 確保模型穩定性
-

報告完成日期: 2026年4月29日